

Convolutional Neural Networks (CNN)

Course: Machine Learning | CSE 411
Green University of Bangladesh

Prepared by:

Md. Sharifur Rahman
Lecturer, Dept. of CSE
Green University of Bangladesh

Topics Covered:

- | | |
|-------------------------------------|------------------------------------|
| 1. CNN Architecture Overview | 5. Output Dimension Calculation |
| 2. Input Layer & RGB Representation | 6. Multi-Channel Convolution |
| 3. Convolution Operation & Kernel | 7. Multiple Filters & Feature Maps |
| 4. Zero-Padding & Stride | 8. ReLU Activation Function |

1. CNN Architecture Overview

A **Convolutional Neural Network (CNN)** is a specialised deep-learning architecture designed to process data with a grid-like topology — most notably images. CNNs exploit **local connectivity** and **weight sharing** through learnable filters, enabling them to automatically detect spatial hierarchies of features (edges → shapes → objects) with far fewer parameters than a fully-connected network.

Key Insight

- Shallow layers detect low-level features: edges, corners, colour blobs.
- Middle layers detect mid-level features: textures, curves, simple shapes.
- Deep layers detect high-level features: faces, wheels, entire objects.

Stage	Components	Purpose
Feature Extraction	Conv + ReLU + Pooling (repeated)	Learn spatial patterns from pixels
Classification	Flatten → FC Layer → Softmax	Map features to class probabilities

A typical image classifier (e.g., cat vs. dog) stacks several Convolution+ReLU+Pooling blocks for feature extraction, then flattens the resulting volume and passes it through fully-connected layers ending in a **Softmax** that outputs a probability for each class.

2. Input Layer — Image Representation

Every image is stored as a **3-D tensor: Width × Height × Channels**. A colour image using the RGB model has **3 channels** (Red, Green, Blue). A grayscale image has 1 channel. Each value is an integer in [0, 255] representing pixel intensity.

Example: A 5×5 RGB colour image

The tensor is **5 × 5 × 3**, holding three 5×5 matrices of pixel values.

R Channel					G Channel					B Channel				
220	210	195	185	170	110	108	106	104	102	40	45	50	55	60
205	195	182	172	160	106	104	102	100	98	44	49	54	59	64
190	180	168	158	145	102	100	98	96	94	48	53	58	63	68
175	165	152	142	130	98	96	94	92	90	52	57	62	67	72
160	150	138	128	116	94	92	90	88	86	56	61	66	71	76

Fig: Three colour channels of a 5×5 RGB image

Key Formula

Input Tensor Shape = Width x Height x Channels

Example: A 128x128 colour image → tensor 128 x 128 x 3 (49,152 values)

A 28x28 grayscale image → tensor 28 x 28 x 1 (784 values)

3. Convolution Operation

The **convolution operation** slides a small learnable matrix called a **kernel** (or filter) across the input. At every position the kernel occupies, corresponding values are multiplied element-wise and summed, producing a single entry in the output **feature map**.

Output Size Formula

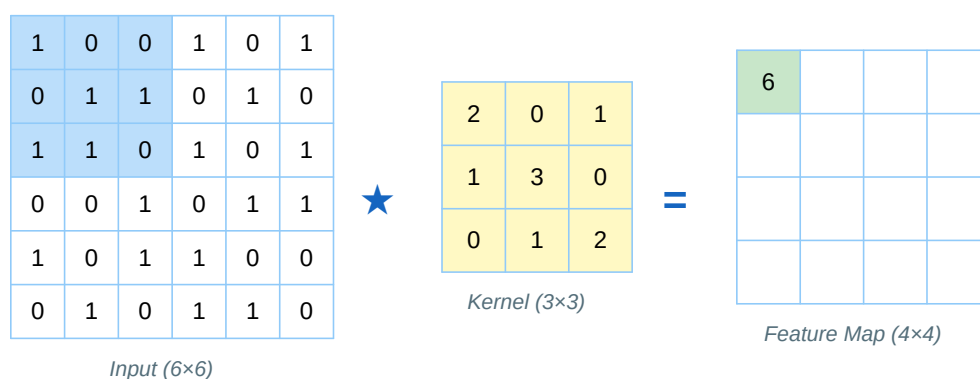
For an input of size M, kernel size F, padding P, and stride S:

$$O = (M - F + 2P) / S + 1$$

Symbol	Meaning	Typical Values
M	Input spatial size (height or width)	28, 64, 224
F	Kernel / filter size	3, 5, 7
P	Zero-padding	0, 1, 2
S	Stride (step size of kernel)	1 or 2
O	Output feature map size	Computed

Worked Example: 6×6 Input, 3×3 Kernel, P=0, S=1

Output size: $O = (6 - 3 + 0) / 1 + 1 = 4 \rightarrow 4 \times 4$ feature map



$$y_{11} = (1 \times 2) + (0 \times 0) + (0 \times 1) + (0 \times 1) + (1 \times 3) + (1 \times 0) + (1 \times 0) + (1 \times 1) + (0 \times 2) = 2 + 0 + 0 + 0 + 3 + 0 + 0 + 1 + 0 = 6$$

Convolution Steps

Step	Action
1. Prepare Input	Arrange pixel values in a grid (matrix).
2. Position Kernel	Place kernel at top-left; it will slide over every valid position.
3. Element-wise Multiply	Multiply each kernel weight by the overlapping input value.
4. Sum Products	Add all products $\rightarrow$ one scalar output value.
5. Add Bias	Add a learned bias term to the scalar.
6. Fill Feature Map	Place the scalar at the corresponding output position.

7. Repeat per Filter

With K filters, produce K feature maps in parallel.

4. Cross-Correlation and Bias

In practice CNNs apply **cross-correlation** rather than strict convolution (the kernel is not flipped). Because the kernel weights are learned, the network simply learns the appropriate flipped weights automatically, so the distinction is immaterial in practice.

Worked Example: 4×4 Input, 3×3 Sharpening Kernel, Bias = 4

Input (4×4)				Kernel (3×3)		
1	2	3	4	0	-1	0
5	6	7	8	-1	5	-1
9	10	11	12	0	-1	0
13	14	15	16			

Position (1,1) — top-left 3×3 patch:

$$(1 \times 0) + (2 \times -1) + (3 \times 0) + (5 \times -1) + (6 \times 5) + (7 \times -1) + (9 \times 0) + (10 \times -1) + (11 \times 0) + 4 = 0 - 2 + 0 - 5 + 30 - 7 + 0 - 10 + 0 + 4 = \mathbf{10}$$

Position (1,2) — shifted one step right:

$$(2 \times 0) + (3 \times -1) + (4 \times 0) + (6 \times -1) + (7 \times 5) + (8 \times -1) + (10 \times 0) + (11 \times -1) + (12 \times 0) + 4 = 0 - 3 + 0 - 6 + 35 - 8 + 0 - 11 + 0 + 4 = \mathbf{11}$$

What is a Bias Term?

Bias (b) is a single learnable value added after the dot product:

$$y = w \cdot x + b$$

It allows the neuron to shift its activation independently of the input.

Without bias, all outputs would be forced through the origin.

5. Padding

Padding adds extra rows and columns (usually zeros) around the border of the input before applying convolution.

Purpose	Explanation
Preserve spatial size	Without padding a 3×3 kernel on a 6×6 input gives a 4×4 output. With P=1 the output stays 6×6 — called "same" padding.
Preserve border information	Corner and edge pixels are used fewer times without padding, causing information loss at boundaries.

Zero-Padding Example: 5×5 image, P=1, F=3, S=1

After padding, the input becomes 7×7. Output size:

$$O = (5 - 3 + 2 \times 1) / 1 + 1 = 5$$

The output feature map has the **same spatial dimensions (5×5)** as the original input.

Padded 7×7 input (blue = zero padding, yellow = original):

0	0	0	0	0	0	0
0	45	80	120	95	60	0
0	70	110	155	130	85	0
0	90	135	180	150	100	0
0	65	100	145	115	75	0
0	50	85	125	100	65	0
0	0	0	0	0	0	0

Fig: Zero-padding applied to a 5×5 image with P=1

6. Output Dimension Calculation

The output volume of a convolutional layer has three dimensions. Height and width are each computed from the same formula; depth equals the number of filters K .

Dimension	Formula
Output Height (OH)	$OH = \text{floor}((H + 2P - F) / S) + 1$
Output Width (OW)	$OW = \text{floor}((W + 2P - F) / S) + 1$
Output Depth (OD)	$OD = K$ (number of filters)

Worked Example

Input volume: **64×64×3** Filter size: **5×5** Padding: **P=2** Stride: **S=1** Filters: **K=16**

Dimension	Calculation	Result
Output Height	$OH = \text{floor}((64+2\times2-5)/1)+1 = \text{floor}(63/1)+1$	64
Output Width	$OW = \text{floor}((64+2\times2-5)/1)+1$	64
Output Depth	$OD = K = 16$	16
Final Shape	64 × 64 × 16	✓

Intuition

With $P=2$ and $F=5$, the spatial size is preserved ($64 \rightarrow 64$). This is "same" padding.

The depth grows from 3 (RGB input) to 16 (one feature map per filter).

Each feature map captures a different pattern: edges, textures, blobs, etc.

7. Multi-Channel Convolution & Multiple Filters

When the input has C channels, each kernel also has depth C . Convolution is applied independently per channel; the C results are summed (plus bias) to yield a single 2-D feature map. Using K different kernels produces K feature maps stacked into the output volume.

Formula for one output value:

$$y = (K_1 \blacksquare I_1) + (K_2 \blacksquare I_2) + \dots + (K_C \blacksquare I_C) + b$$

Numerical Example — RGB Input (first position)

Channel	Input Patch (3×3)	Kernel (3×3)	Result
Red (R)	220, 210, 195 205, 195, 182 190, 180, 168	-1, 0, 1 0, 1, 0 1, 0, -1	42
Green (G)	110, 108, 106 106, 104, 102 102, 100, 98	1, 0, 0 1, -1, -1 1, 0, -1	-8
Blue (B)	40, 45, 50 44, 49, 54 48, 53, 58	0, 1, 1 0, 1, 0 1, -1, 1	115

$$\text{Output} = 42 + (-8) + 115 + \text{Bias}(3) = 152$$

Multiple Filters — What They Detect

Filter	Detects
Filter 1 (Horizontal Sobel)	Top / bottom intensity gradients (horizontal edges)
Filter 2 (Vertical Sobel)	Left / right gradients (vertical edges)
Filter 3 (Laplacian)	Regions of rapid intensity change (blob / sharpness)
Filter K (Learned)	Abstract high-level patterns specific to the task

8. ReLU Activation Function

After every convolution a **non-linear activation function** is applied element-wise to the feature map. The standard choice in CNNs is **Rectified Linear Unit (ReLU)**.

$$\text{ReLU}(x) = \max(0, x)$$

All negative values in the feature map become **zero**; positive values pass through unchanged. This introduces the non-linearity that lets the network learn complex, non-linear decision boundaries.

Why ReLU?

Property	Detail
Fast computation	One threshold comparison — no costly exponentials.
Sparse activations	Negatives → 0, producing sparse and efficient representations.
No vanishing gradient	Gradient = 1 for positive inputs (unlike sigmoid/tanh which saturate).
Faster training	Networks converge significantly faster with ReLU.

ReLU Example: 4×4 Feature Map

Before ReLU					After ReLU			
5	12	-7	3	→ ReLU →	5	12	0	3
-4	-18	8	6		0	0	8	6
-2	9	15	-1		0	9	15	0
7	44	-5	60		7	44	0	60

Fig: Feature map before and after ReLU — red cells (negative) become green (zero)

Key Takeaway — ReLU

- ReLU(x) = max(0, x)
- Negative values → 0 (blocked)
 - Positive values → unchanged (passed through)
 - Non-linearity enables the CNN to learn complex, non-linear patterns.

9. Summary

Component	Role	Key Note
Input Layer	Stores raw pixel values as a 3-D tensor	Shape: $W \times H \times C$
Convolution Layer	Extracts spatial features via sliding kernels	$O = (M - F + 2P) / S + 1$
Kernel / Filter	Small learnable weight matrix	Size: $F \times F \times C$
Feature Map	Output of one kernel applied to input	Encodes pattern presence
Bias	Learnable offset after convolution	$y = w \cdot x + b$
Zero-Padding	Adds zeros around input border	Preserves spatial size
Stride	Step size of sliding kernel	Larger $S \rightarrow$ smaller O
ReLU	Removes negative activations	$\max(0, x)$
Multiple Filters	K filters $\rightarrow$ depth- K output volume	Each detects a different feature

CNN Pipeline at a Glance

Input Image

$\rightarrow$ [Convolution + ReLU] $\rightarrow$ Pooling

$\rightarrow$ [Convolution + ReLU] $\rightarrow$ Pooling

$\rightarrow$ Flatten $\rightarrow$ Fully Connected $\rightarrow$ Softmax

$\rightarrow$ Class Probabilities (e.g. Cat: 0.92, Dog: 0.06, Bird: 0.02)